

Data Visualization and Insights Extraction

- Univariate Analysis (Categorical or Continuous)
- Multivariate Analysis (Categorical or Continuous)
- Faceting of the Plots
- Advanced Plots and Charts

Load the Required Libraries

In [1]:

```
import pandas as pd          ## importing and manipulating df
import numpy as np          ## Numeric manipulation
import matplotlib.pyplot as plt  ## For data visualization
import seaborn as sns       ## For data visualization
from pandas.plotting import scatter_matrix  ## For data visualization
from mpl_toolkits.mplot3d import Axes3D    ## For data visualization
import warnings              ## Turn off warnings
warnings.filterwarnings('ignore')          ## Actual turning off of the warnings
```

Set up Data Visualization Style

In [2]:

```
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = [10, 6]
plt.rcParams['font.size'] = 10
```

Data Importation

In [3]:

```
df = pd.read_csv('Clean_GSSsubset.csv')
df.head()
```

Out[3]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	HIGH SCHOOL	26722.56	NEVER MARRIED	18	73.2	86.3	35
1	2	MALE	HIGH SCHOOL	36544.91	MARRIED	35	69.9	95.3	47
2	3	FEMALE	HIGH SCHOOL	59587.14	DIVORCED	28	70.1	60.9	16
3	4	FEMALE	GRADUATE	49586.54	MARRIED	43	61.9	76.0	44
4	5	MALE	HIGH SCHOOL	48899.47	MARRIED	48	65.8	58.9	44

Display of Basic Information

In [25]:

```
### Dimension
print(f"Display the Dimension of the Data:{df.shape}")
```

Display the Dimension of the Data:(3000, 9)

In [26]:

```
### Display the Column Names
```

```
print(f"Display the Column Names: {list(df.columns)}")
```

Display the Column Names: ['id', 'sex', 'degree', 'income', 'marital', 'age', 'height', 'weight', 'hrswrk']

In [27]:

```
#### Display the Type of Each Variable
```

```
df.info()
```

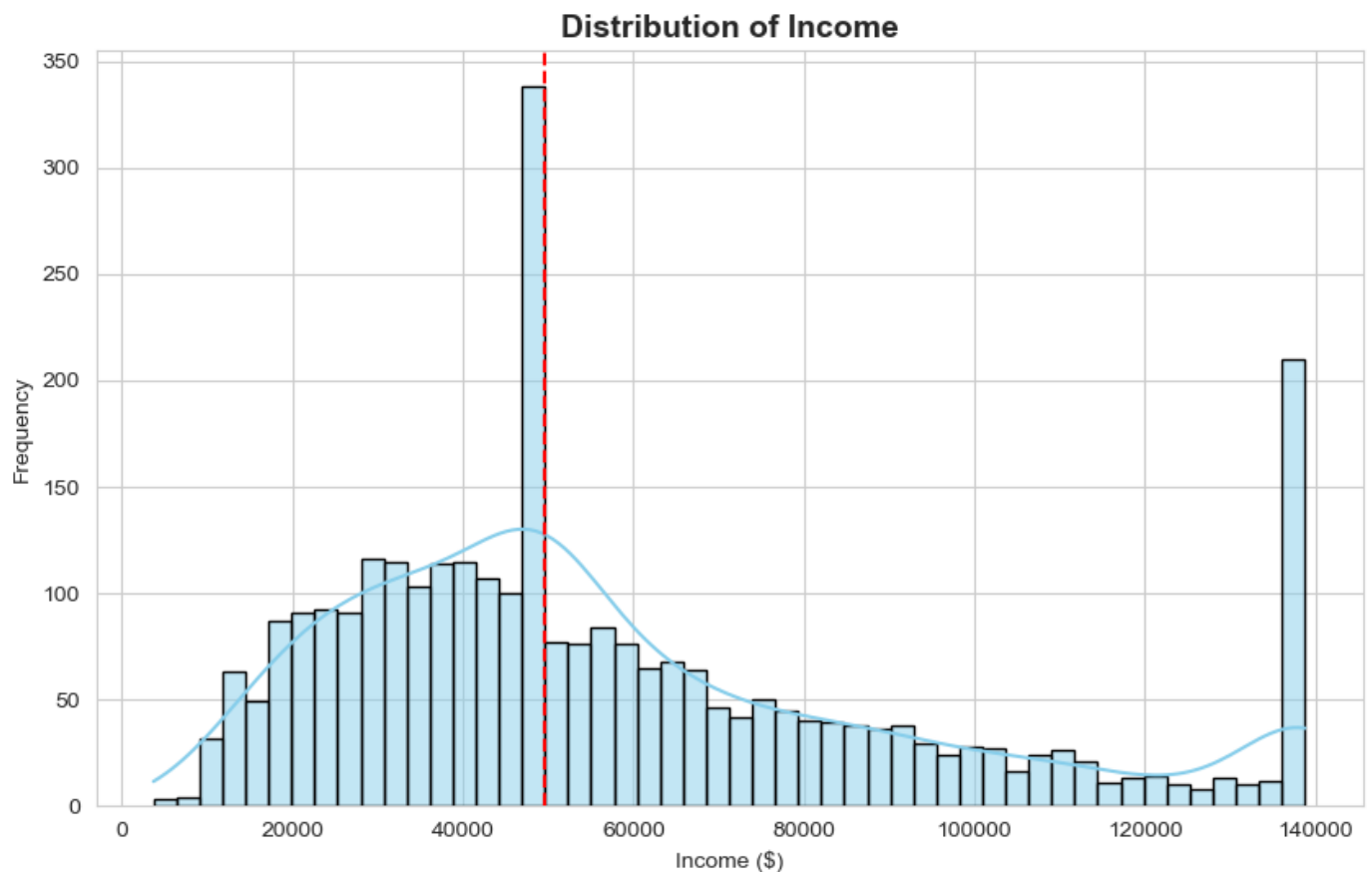
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   id          3000 non-null   int64  
 1   sex         3000 non-null   object  
 2   degree      3000 non-null   object  
 3   income      3000 non-null   float64 
 4   marital     3000 non-null   object  
 5   age         3000 non-null   int64  
 6   height      3000 non-null   float64 
 7   weight      3000 non-null   float64 
 8   hrswrk      3000 non-null   int64  
dtypes: float64(3), int64(3), object(3)
memory usage: 211.1+ KB
```

2. Univariate Analysis

2.1. Histogram for the Distribution of Income

In [32]:

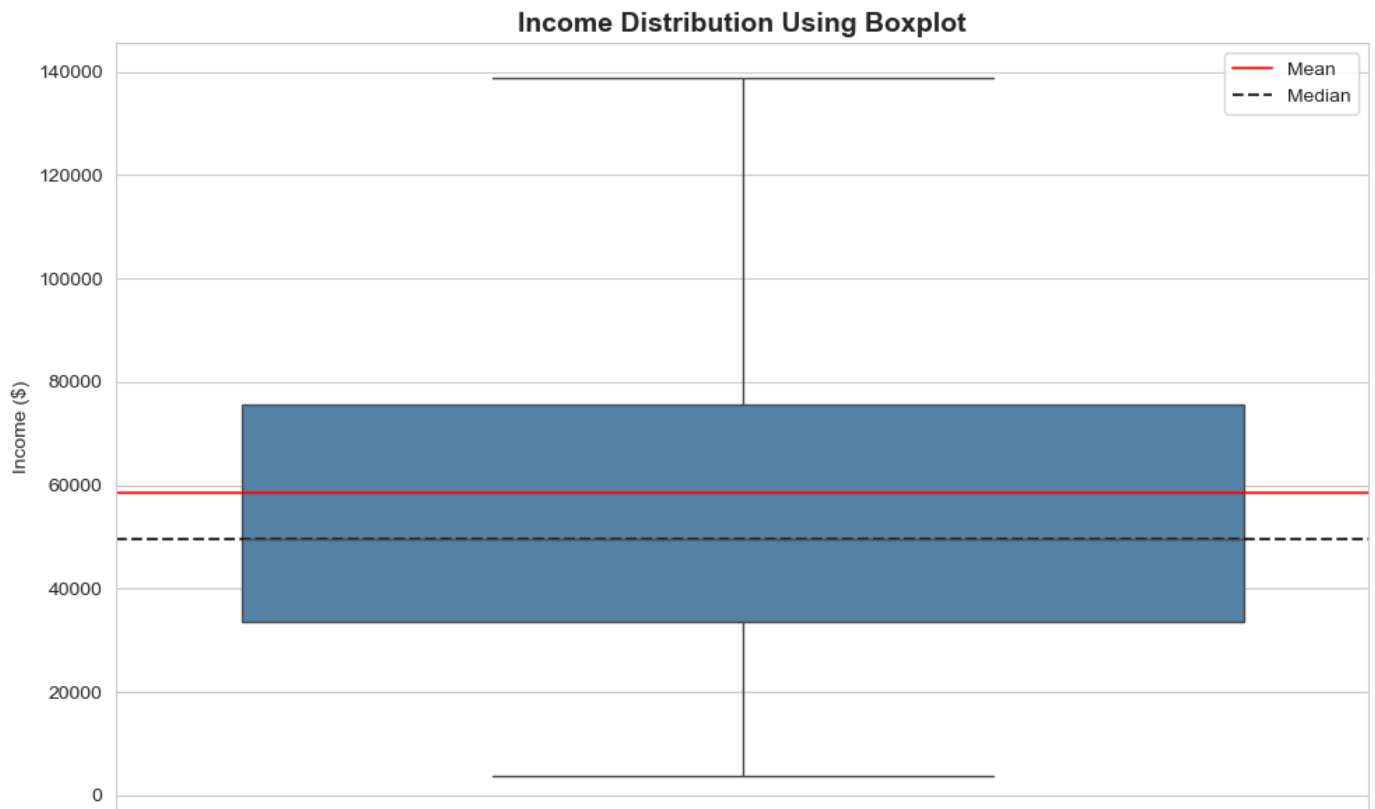
```
plt.figure(figsize = (10,6))
sns.histplot(df['income'], bins = 50, kde = True,
             color = 'skyblue', edgecolor = 'black')
plt.title('Distribution of Income',
         fontsize = 14, fontweight = 'bold')
plt.xlabel('Income ($)')
plt.ylabel('Frequency')
plt.axvline(df['income'].median(), color = 'red',
           linestyle = '--',
           label = f'Median: ${df["income"].median():.0f}')
plt.show()
```



2.2. Income Distribution from a Boxplot

In [40]:

```
plt.figure(figsize = (10, 6))
sns.boxplot(y = df['income'], color = 'steelblue')
plt.title("Income Distribution Using Boxplot", fontsize = 14,
          fontweight = 'bold')
plt.ylabel('Income ($)')
plt.axhline(df['income'].mean(), color = 'red',
            linestyle = '-', alpha = 0.8, label = 'Mean')
plt.axhline(df['income'].median(), color = 'black',
            linestyle = '--', alpha = 0.8, label = 'Median')
plt.legend()
plt.tight_layout()
plt.show()
```



Identification of Outliers Using Boxplot

In [42]:

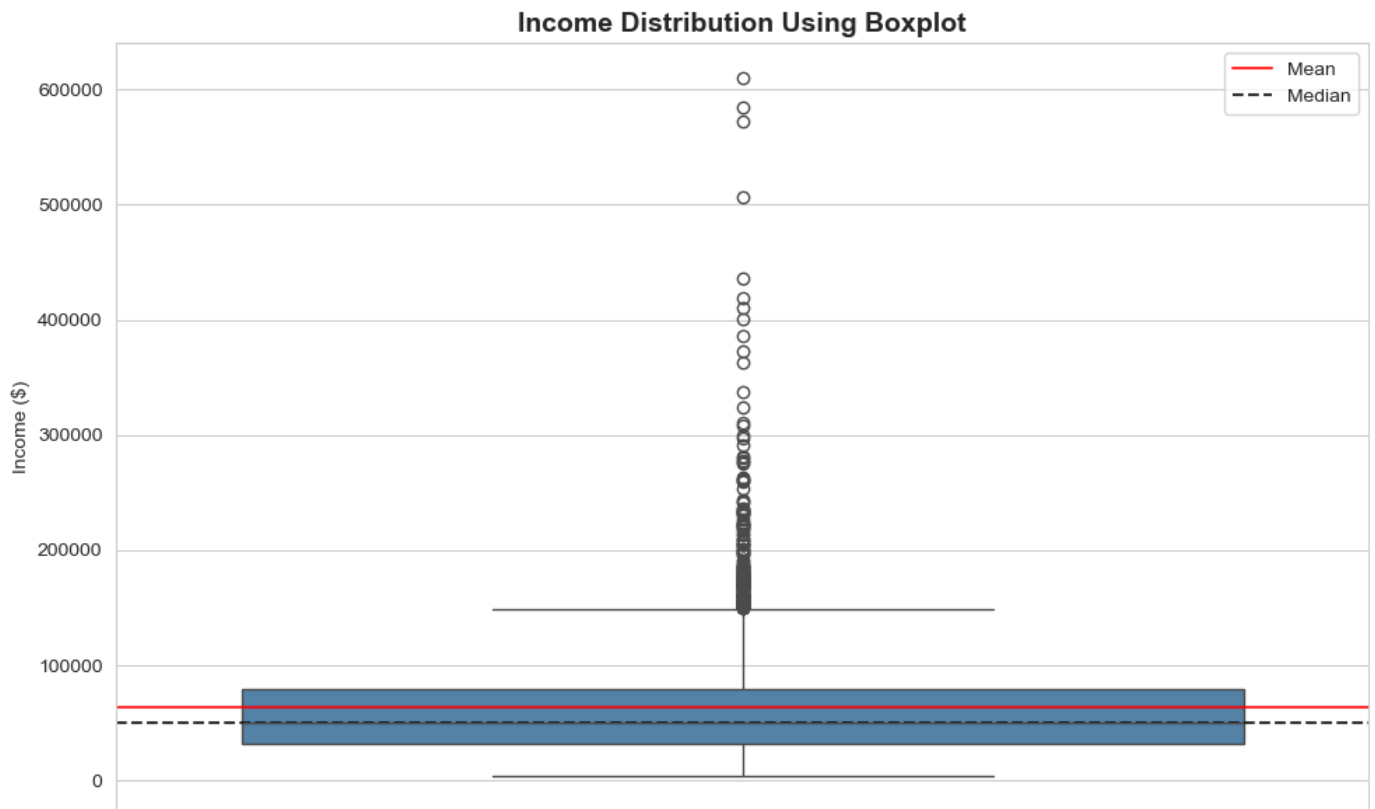
```
df2 = pd.read_csv("GSSsubset1.csv")
df2.head()
```

Out[42]:

	id	sex	degree	income	marital	age	height	weight	hswrk
0	1	MALE	HIGH SCHOOL	26722.56	NEVER MARRIED	18	73.2	86.3	35.0
1	2	MALE	NaN	36544.91	MARRIED	35	69.9	95.3	47.0
2	3	FEMALE	HIGH SCHOOL	59587.14	DIVORCED	28	70.1	60.9	16.0
3	4	FEMALE	GRADUATE	NaN	MARRIED	43	61.9	76.0	44.0
4	5	MALE	HIGH SCHOOL	48899.47	MARRIED	48	65.8	58.9	44.0

In [43]:

```
plt.figure(figsize = (10, 6))
sns.boxplot(y = df2['income'], color = 'steelblue')
plt.title("Income Distribution Using Boxplot", fontsize = 14,
          fontweight = 'bold')
plt.ylabel('Income ($)')
plt.axhline(df2['income'].mean(), color = 'red',
            linestyle = '-', alpha = 0.8, label = 'Mean')
plt.axhline(df2['income'].median(), color = 'black',
            linestyle = '--', alpha = 0.8, label = 'Median')
plt.legend()
plt.tight_layout()
plt.show()
```



2.3. Age Distribution Using Histogram and Violin Plots

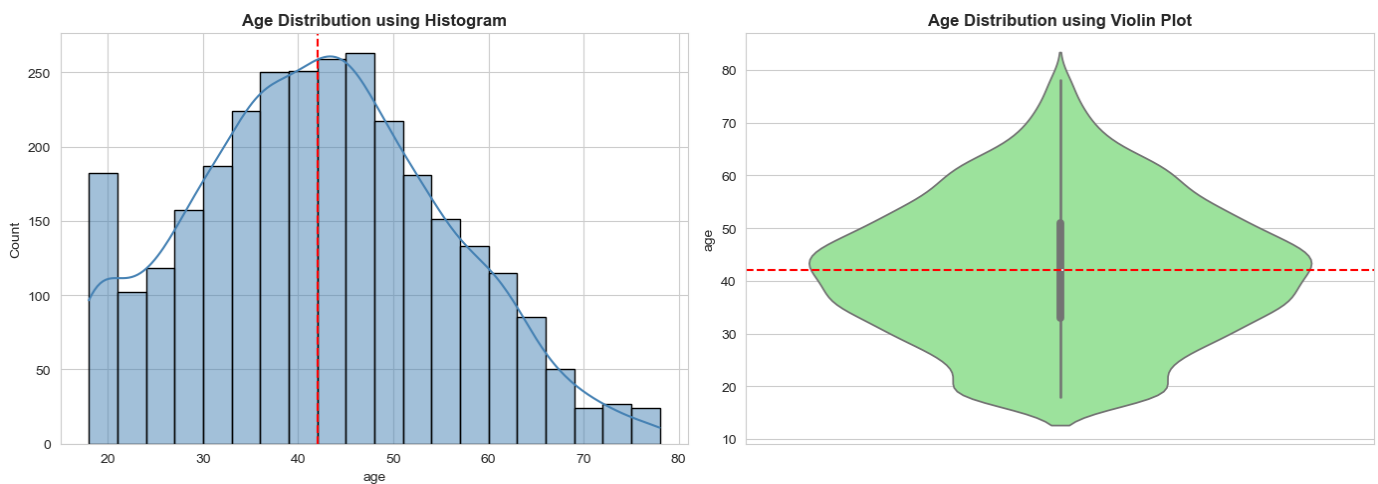
In [47]:

```
fig, axes = plt.subplots(1, 2, figsize = (14, 5))

sns.histplot(df['age'], bins = 20, kde = True, ax=axes[0],
             color = 'steelblue', edgecolor = 'black')
axes[0].set_title('Age Distribution using Histogram', fontweight = 'bold')
axes[0].axvline(df['age'].mean(), color = 'red', linestyle = '--')

sns.violinplot(y = df['age'], ax = axes[1], color = 'lightgreen')
axes[1].set_title("Age Distribution using Violin Plot", fontweight = 'bold')
axes[1].axhline(df['age'].mean(), color = 'red', linestyle = '--')

plt.tight_layout()
plt.show()
```



2.4. Joint Distribution Plot - Height and Weight

In [54]:

```
joint_plot = sns.jointplot(data = df, x = 'height', y = 'weight',
                           kind = 'scatter', alpha = 0.8, color = 'purple',
                           marginal_kws = dict(bins = 50, fill = True))
joint_plot.fig.suptitle('A joint Plot of Weight and Height Distribution',
                        fontsize = 14, fontweight = 'bold')

plt.tight_layout()
plt.show()

corr_val = df['height'].corr(df['weight'])
print(f"Height and Weight Correlation: {corr_val:.3f}")
```



Height and Weight Correlation: 0.643

3. Categorical Variables Analysis

- Bar Chart - Gender and Educational Level Distribution
- Pie and Count Plot (Bar Chart) showing the Distribution of Marital Status

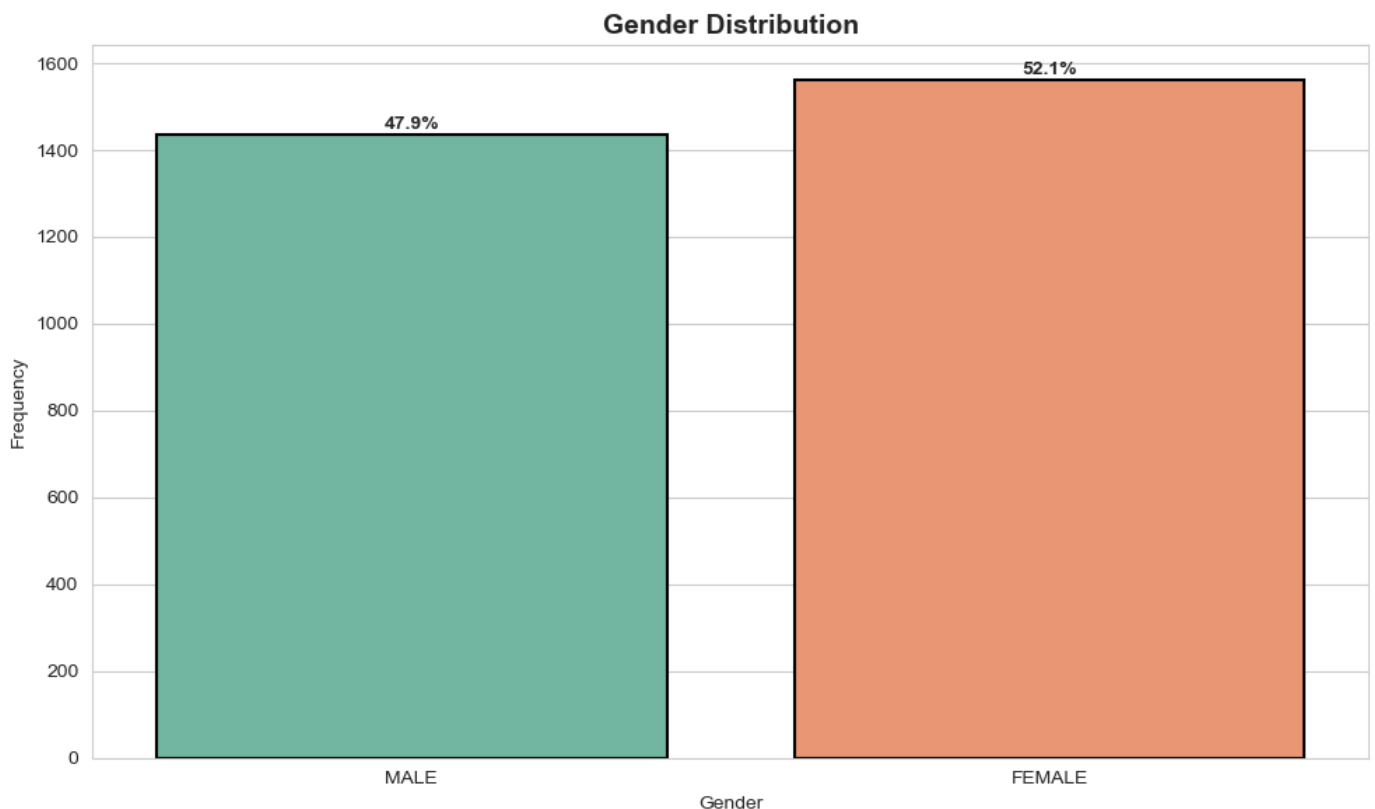
3.1. Bar Plot for Gender Distribution

In [7]:

```
plt.figure(figsize = (10, 6))
ax = sns.countplot(data = df, x = 'sex', palette = 'Set2',
                  edgecolor = 'black', linewidth = 1.5)
plt.title("Gender Distribution", fontsize = 14, fontweight = 'bold')
plt.xlabel("Gender")
plt.ylabel("Frequency")

total = len(df)
for p in ax.patches:
    percentage = f'{100*p.get_height()/total:.1f}%'
    x = p.get_x() + p.get_width()/2
    y = p.get_height() + 10
    ax.annotate (percentage, (x, y), ha = 'center', fontweight = 'bold')

plt.tight_layout()
plt.show()
```

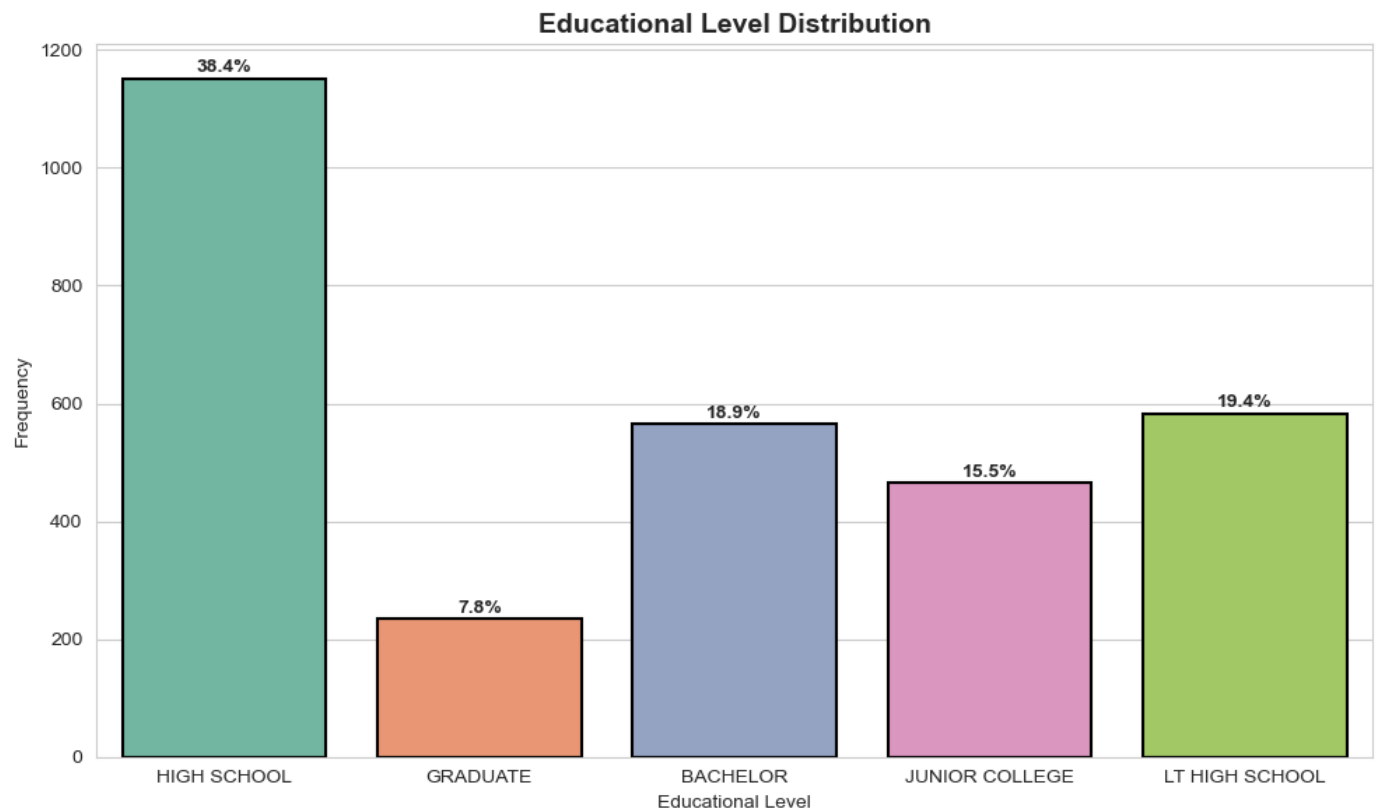


In [9]:

```
plt.figure(figsize = (10, 6))
ax = sns.countplot(data = df, x = 'degree', palette = 'Set2',
                  edgecolor = 'black', linewidth = 1.5)
plt.title("Educational Level Distribution", fontsize = 14, fontweight = 'bold')
plt.xlabel("Educational Level")
plt.ylabel("Frequency")

total = len(df)
for p in ax.patches:
    percentage = f'{100*p.get_height()/total:.1f}%'
    x = p.get_x() + p.get_width()/2
    y = p.get_height() + 10
    ax.annotate (percentage, (x, y), ha = 'center', fontweight = 'bold')
```

```
plt.tight_layout()
plt.show()
```



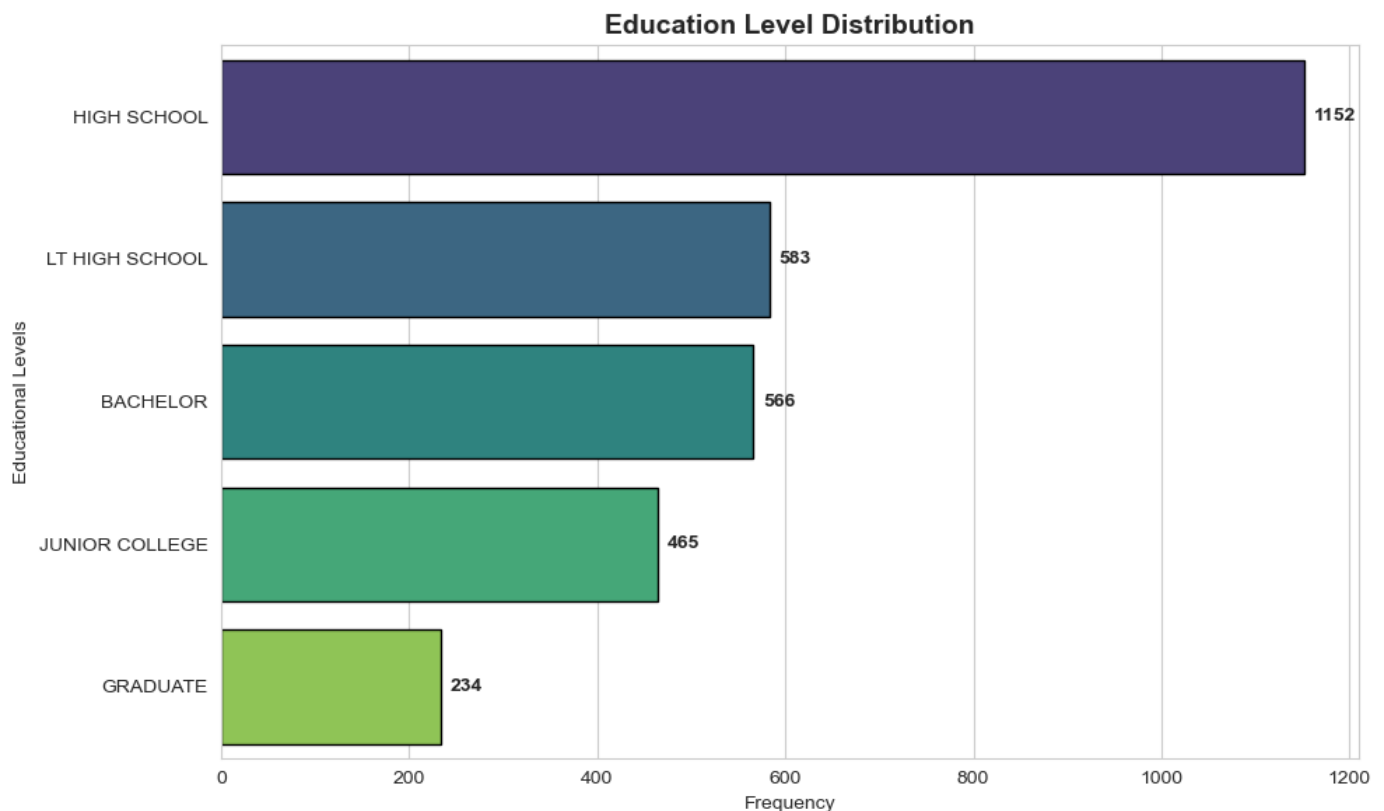
Horizontal Bar Chart for Educational Level

In [15]:

```
plt.figure(figsize = (10, 6))
degree_counts = df['degree'].value_counts()
ax = sns.barplot(x = degree_counts.values, y = degree_counts.index,
                 palette = 'viridis', edgecolor = 'black')
plt.title('Education Level Distribution', fontsize = 14, fontweight = 'bold')
plt.xlabel('Frequency')
plt.ylabel("Educational Levels")

for i, v in enumerate(degree_counts.values):
    ax.text(v+10, i, str(v), va = "center", fontweight = 'bold')

plt.tight_layout()
plt.show()
```

In [32]:

```
plt.figure(figsize = (14, 6))

marital_counts = df['marital'].value_counts()
color = plt.cm.Set3(np.linspace(0, 1, len(marital_counts)))

sns.pie(marital_counts, labels = marital_counts.index,
        autopct = '%1.1f%', colors = color, startangle = 90,
        explode = [0.05]* len(marital_counts))
plt.title('Marital Status - For the Pie Chart', fontweight = 'bold')
plt.tight_layout()
plt.show()
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[32], line 6
      3 marital_counts = df['marital'].value_counts()
      4 color = plt.cm.Set3(np.linspace(0, 1, len(marital_counts)))
----> 6 sns.pie(marital_counts, labels = marital_counts.index,
      7         autopct = '%1.1f%', colors = color, startangle = 90,
      8         explode = [0.05]* len(marital_counts))
      9 plt.title('Marital Status - For the Pie Chart', fontweight = 'bold')
     10 plt.tight_layout()
```

AttributeError: module 'seaborn' has no attribute 'pie'
<Figure size 1400x600 with 0 Axes>

Pie Chart and a Countplot for Marital Status

In [30]:

```
fig, axes = plt.subplots(1, 2, figsize = (14, 6))

marital_counts = df['marital'].value_counts()
```

```

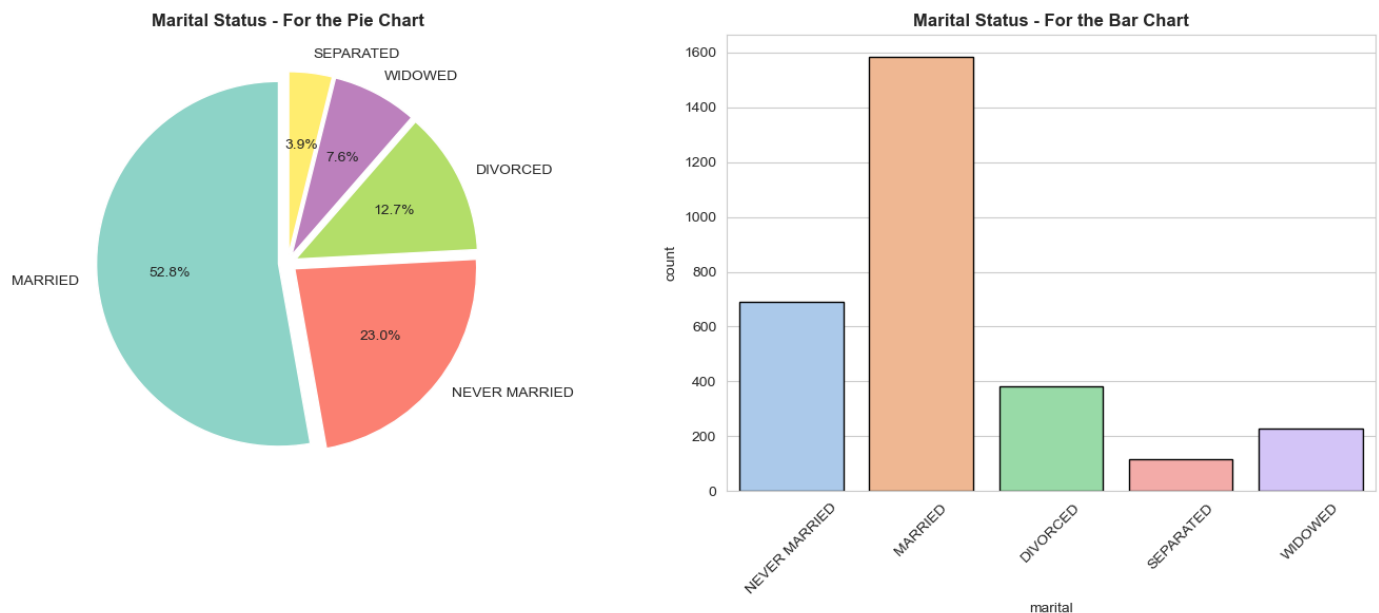
color = plt.cm.Set3(np.linspace(0, 1, len(marital_counts)))

axes[0].pie(marital_counts, labels = marital_counts.index,
            autopct = '%1.1f%%', colors = color, startangle = 90,
            explode = [0.05]* len(marital_counts))
axes[0].set_title('Marital Status - For the Pie Chart', fontweight = 'bold')

sns.countplot(data = df, x = "marital", ax = axes[1],
              palette = 'pastel', edgecolor = 'black')
axes[1].set_title('Marital Status - For the Bar Chart', fontweight = 'bold')
axes[1].tick_params(axis = 'x', rotation = 45)

plt.tight_layout()
plt.show()

```



4. Bivariate Analysis

- Income by Degree or Gender
- violin - Income by Education Level
- Scatter Plot - Income by Age - Hours of work by Income

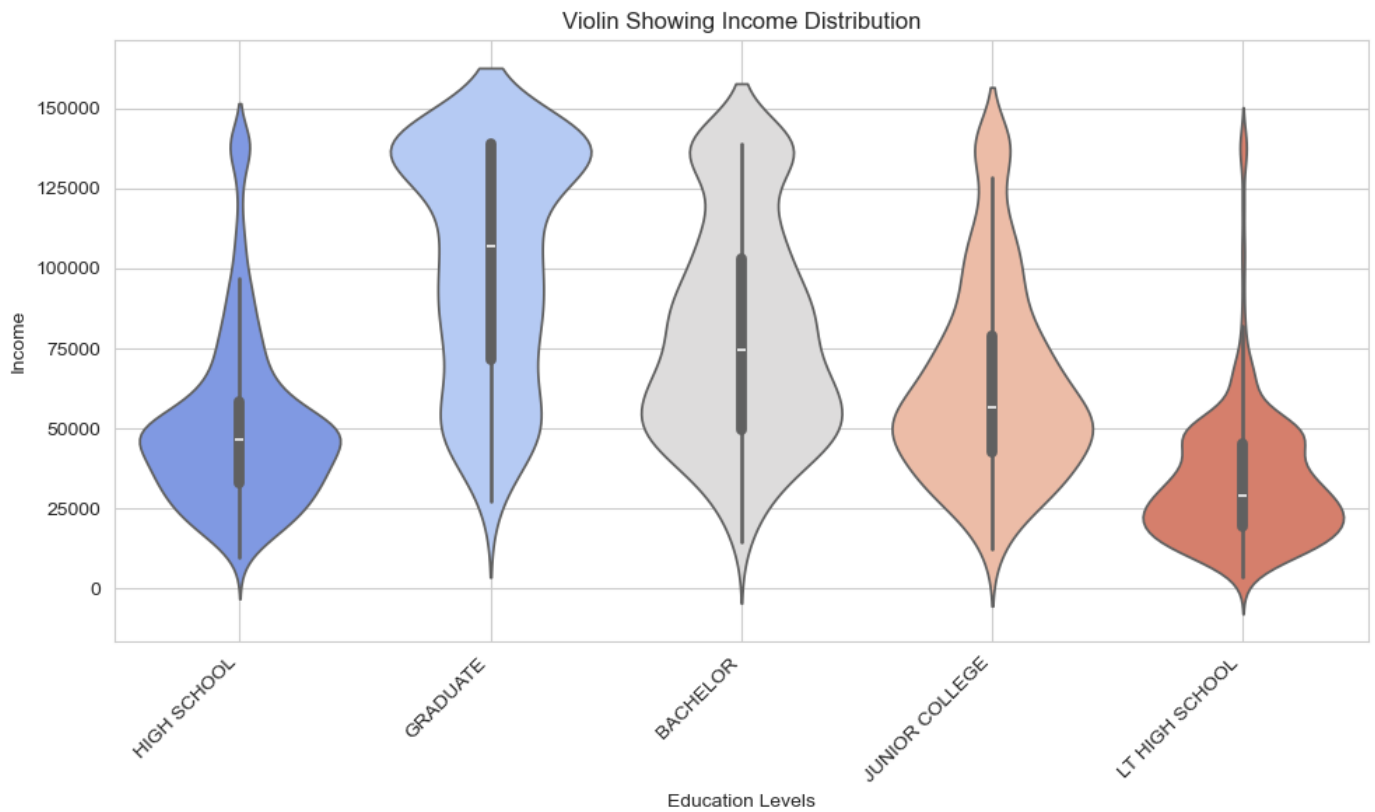
4.1. Income by Education

In [34]:

```

plt.figure(figsize = (10, 6))
sns.violinplot(data= df, x = 'degree', y = 'income',
               palette = 'coolwarm', inner = 'box')
plt.title("Violin Showing Income Distribution")
plt.xlabel("Education Levels")
plt.ylabel("Income")
plt.xticks(rotation = 45, ha = 'right')
plt.tight_layout()
plt.grid(True)
plt.show()

```



4.1 Income by Gender- Bar Chart

In []:

In []:

In []:

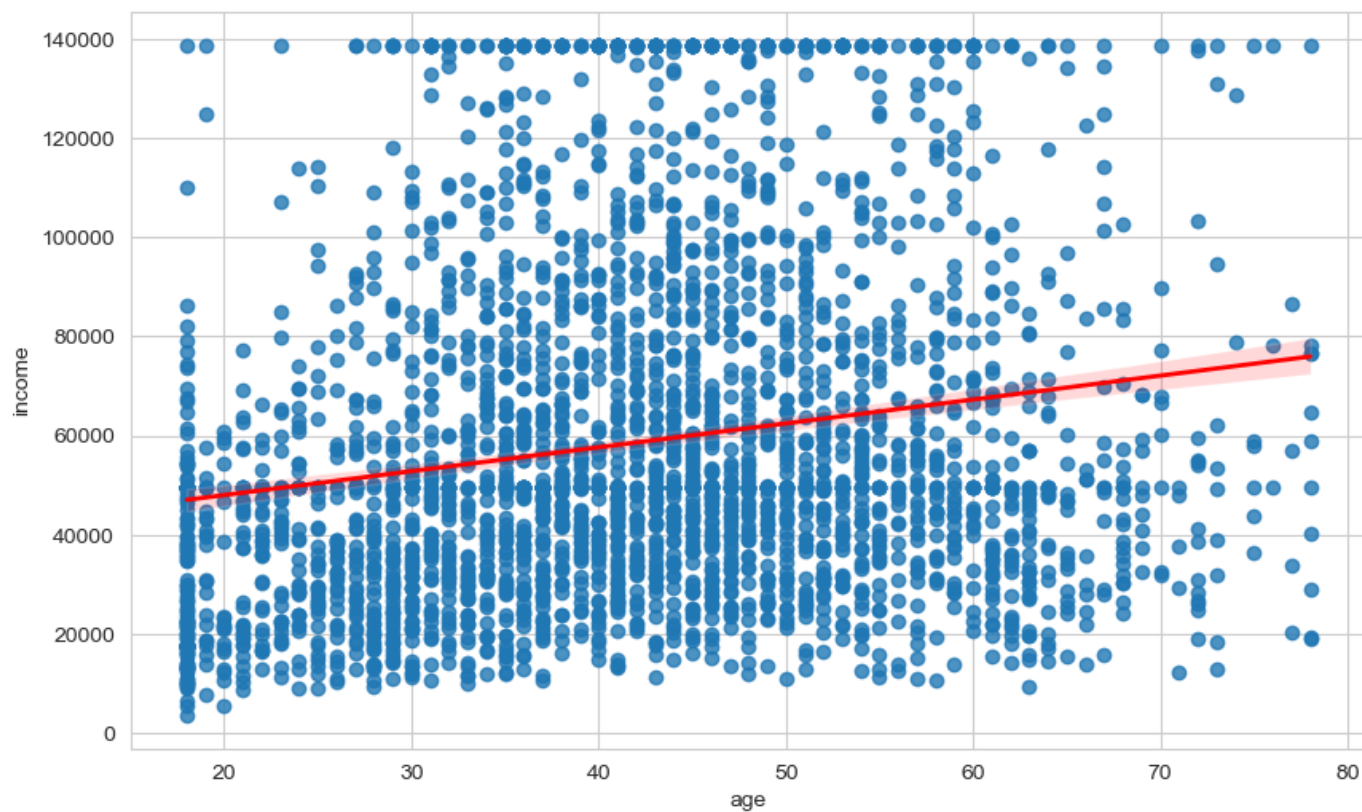
4.2 Income by Education Qualification- Bar Chart

In []:

4.3. Scatter Plot

In [36]:

```
plt.figure(figsize = (10, 6))
sns.regplot(data = df, x = 'age', y = 'income', scatter_kws = {'alpha': 0.8},
            line_kws = {'color': 'red', 'linewidth': 2})
plt.show()
```



In []: